

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

College of Computing and Data Science

SC2006 – Software Engineering

Lab 3 Deliverables

SCSJ Group 3

Name	Matriculation No.
Xu Hanwen	U2522613H
Teo Kok Wei, Oscar	U2410894D
K Eswaran	U2521677A
Mao Yikai	U2520028F
Tan Leong Jun Joseph	U2321339H

Use Case Models

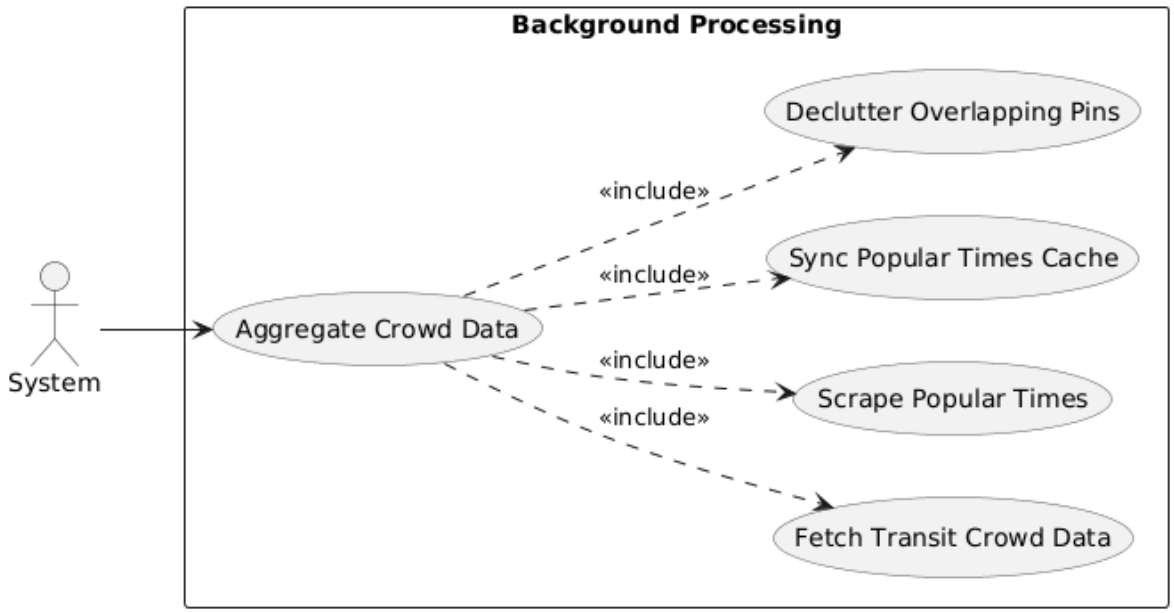


Figure 1. Background Processing (System)

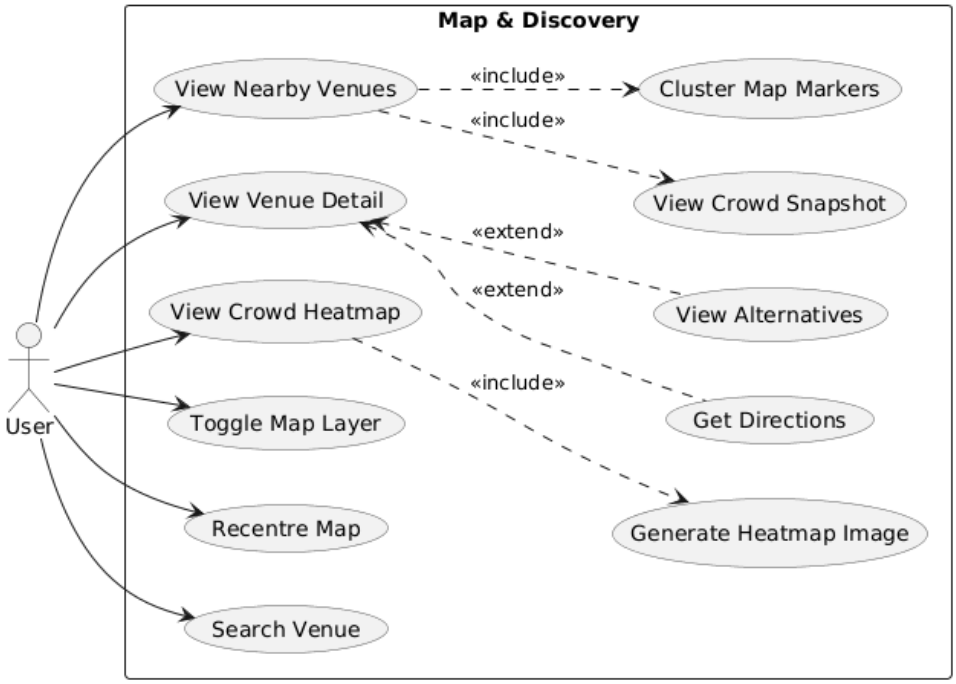


Figure 2. Map & Discovery (User)

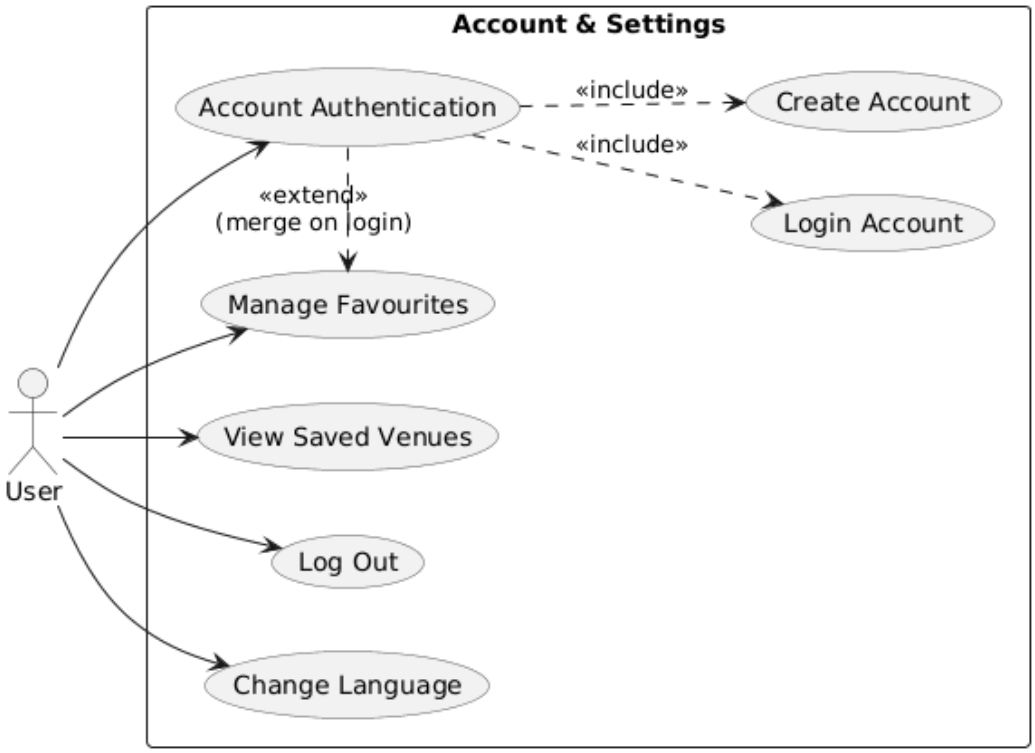


Figure 3. Account & Settings (User)

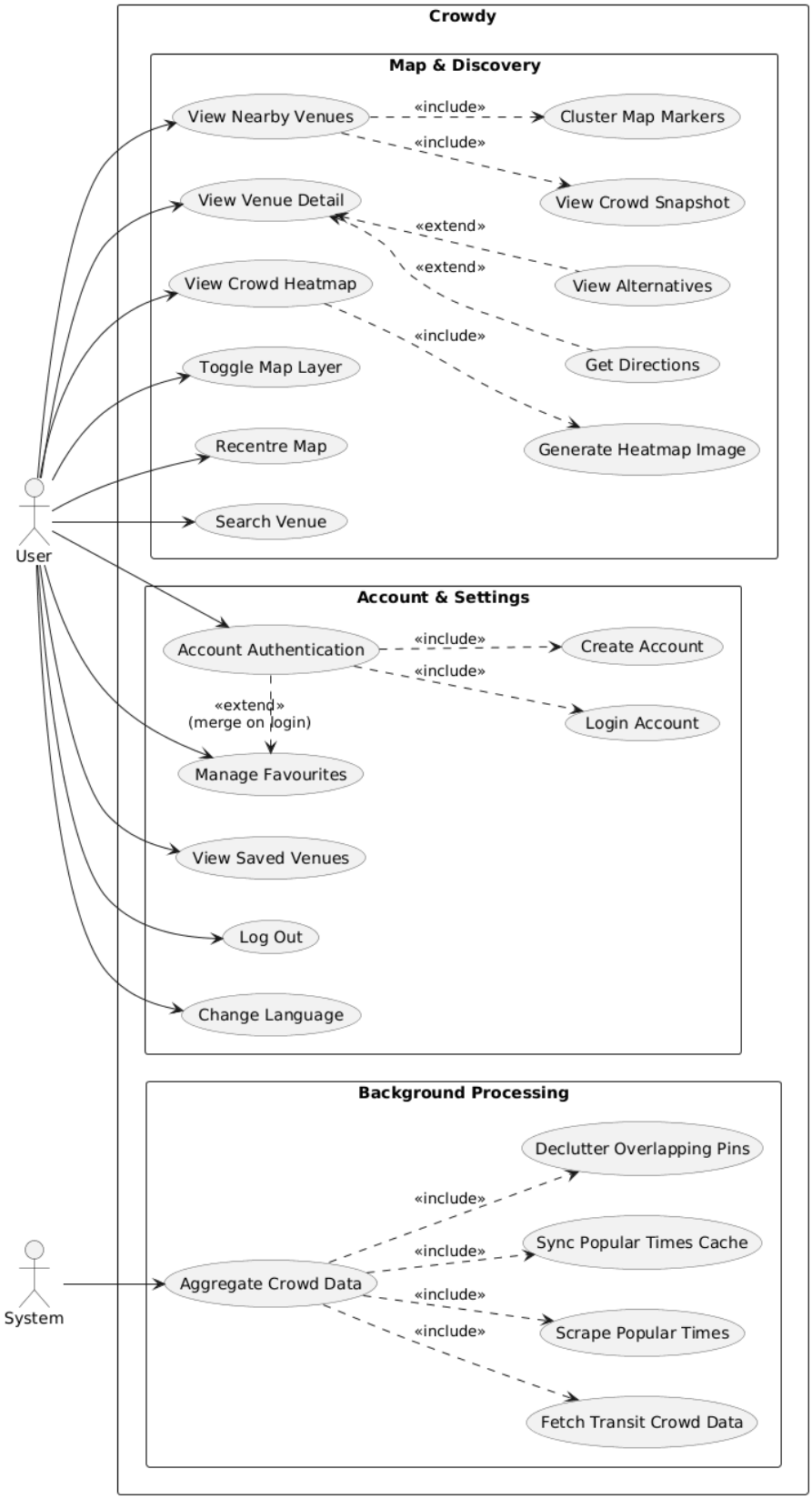


Figure 4. Complete Use Case Diagram

Use Case Descriptions

For Functional Requirement #1 – Background crowd data aggregation

Use Case ID:	#1-1		
Use Case Name:	AggregateCrowdData		
Created By:	Joseph Tan	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	20/10/2026

Actor	System
Description	Periodically collects crowd data from external sources and merges it into one unified venue list.
Preconditions	None (runs on a schedule / cache expiry).
Postconditions	An up-to-date, merged venue list is available to serve users.
Priority	High
Frequency of Use	High (every 5 minutes for transit; each mall's pattern is refreshed weekly)
Flow of Events	1. System checks whether cached transit data has expired (5 min). 2. If expired, invokes FetchTransitCrowdData (#1-2). 3. System checks for malls whose Popular Times pattern is over a week old. 4. If any, invokes ScrapePopularTimes (#1-3) for them one at a time. 5. System merges transit and mall data into one list. 6. System invokes DeclutterOverlappingPins (#1-4) before serving it. 7. System invokes SyncPopularTimesCache (#1-5) on start-up and every 15 minutes.
Alternative Flows	If a source fails, the system keeps serving its last known data: each mall's last pattern, however old, and each station's last reading for up to an hour.
Exceptions	If a source fails for a subset of records, only that subset is skipped.
Includes	FetchTransitCrowdData (#1-2), ScrapePopularTimes (#1-3), DeclutterOverlappingPins (#1-4), SyncPopularTimesCache (#1-5)
Special Requirements	Requires a valid LTA DataMall API account key.
Assumptions	None
Notes/Issues	None

Use Case ID:	#1-2		
Use Case Name:	FetchTransitCrowdData		
Created By:	Joseph Tan	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	20/10/2026

Actor	System
Description	Fetches real-time MRT/LRT crowd density from LTA DataMall.
Preconditions	Cached transit data has expired.
Postconditions	Current crowd levels for all tracked stations are retrieved.
Priority	High
Frequency of Use	High (every 5 minutes)
Flow of Events	1. System calls LTA DataMall's Station Crowd Density API. 2. System maps each station's raw crowd code to a Low/Moderate/High level. 3. System checks each station's real operating hours; a closed station is marked Closed regardless of the raw reading.
Alternative Flows	None
Exceptions	If the API is unreachable, each station keeps its last reading for up to one hour, then shows Unavailable.
Includes	None
Special Requirements	None
Assumptions	None
Notes/Issues	LTA's feed has no notion of "closed" — derived separately from real per-station first/last train times.

Use Case ID:	#1-3		
Use Case Name:	ScrapePopularTimes		
Created By:	Joseph Tan	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	20/10/2026

Actor	System
Description	Retrieves live "Popular Times" crowd data for a mall/venue, since no official API exists for it.
Preconditions	A mall's pattern is over a week old, or the user opened a mall whose pattern is over a week old.
Postconditions	The venue's current crowd % and weekly pattern are updated.
Priority	High
Frequency of Use	Weekly per mall, one mall about every minute.
Flow of Events	1. System opens Google Maps for the venue in a Chrome profile signed in to the project's Google account. 2. System opens the venue's place page. 3. System scrolls to load the Popular Times chart. 4. System extracts the full week's hourly pattern and opening hours. 5. System caches the result and uploads it to the shared database.
Alternative Flows	If Google shows its signed-out "limited view" (no Popular Times), scraping pauses and the server log asks for the account to be signed in again.
Exceptions	If the chart never loads, the venue falls back to its last cached reading.
Includes	None
Special Requirements	Headless Chrome with a signed-in Google profile, on one designated machine.
Assumptions	None
Notes/Issues	Fragile, since it depends on an unofficial page layout. Since October 2026, Google only shows Popular Times to signed-in visitors.

Use Case ID:	#1-4		
Use Case Name:	DeclutterOverlappingPins		
Created By:	Joseph Tan	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	21/10/2026

Actor	System
Description	Repositions venues sharing a near-identical map location so each remains individually visible and selectable.
Preconditions	A merged list of venues with coordinates is available.
Postconditions	No two venues occupy the same small map area.
Priority	Medium
Frequency of Use	High
Flow of Events	1. System groups venues by rounding coordinates to a small grid cell. 2. For each group with more than one venue, system computes the shared centre point. 3. System arranges the group's venues in a small circle around that centre.
Alternative Flows	None
Exceptions	None
Includes	None
Special Requirements	None
Assumptions	None
Notes/Issues	Distinct from ClusterMapMarkers (#2-2), which is a zoom-dependent display grouping, not a coordinate fix.

Use Case ID:	#1-5		
Use Case Name:	SyncPopularTimesCache		
Created By:	Joseph Tan	**Last Updated By:**	Joseph Tan
Date Created:	20/10/2026	**Date Last Updated:**	20/10/2026

Actor	System
Description	Keeps each backend's Popular Times cache in step with the team's shared copy in the database, so a pattern scraped on one machine is available to every backend.
Preconditions	Database connection details are configured.
Postconditions	The local cache and the shared database both hold the newest pattern for every mall.
Priority	Medium
Frequency of Use	On start-up, every 15 minutes, and before each scraping session
Flow of Events	1. System loads all stored patterns from the shared database. 2. For each mall whose shared pattern is newer than the local one, system copies it into the local cache. 3. For each mall whose local pattern is newer, system uploads it to the database. 4. Whenever a new pattern is scraped, system also uploads it immediately.
Alternative Flows	If the database is unreachable, system logs it once and keeps working from its local cache; the next sync retries.
Exceptions	A failed scrape (no pattern) never overwrites a stored pattern.
Includes	None
Special Requirements	MySQL database (hosted on Aiven) with its SSL certificate.
Assumptions	Only one backend scrapes; the others only read from the database.
Notes/Issues	None

For Functional Requirement #2 – View nearby venues

Use Case ID:	#2-1		
Use Case Name:	ViewNearbyVenues		
Created By:	Joseph Tan	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	20/10/2026

Actor	User
Description	Displays a map and list of nearby venues (transit stations and malls) with current crowd levels.
Preconditions	User has granted location permission, or the system falls back to a default location.
Postconditions	Map and list show venues near the user, each with a crowd level.
Priority	High
Frequency of Use	High
Flow of Events	1. App opens. 2. System requests current location. 3. System retrieves nearby venues via AggregateCrowdData (#1-1). 4. System shows the crowd snapshot card, ViewCrowdSnapshot (#2-5), above the list. 5. System displays venues as colour-coded pins on the map. 6. System displays the same venues as a distance-sorted list below the map. 7. As the user pans/zooms, the list updates to match the visible area.
Alternative Flows	If location permission is denied, the map centres on a default location (Marina Bay).
Exceptions	If no venues are available, an empty-state message is shown.
Includes	AggregateCrowdData (#1-1), ClusterMapMarkers (#2-2), ViewCrowdSnapshot (#2-5)
Special Requirements	Device location services (GPS).
Assumptions	None
Notes/Issues	The Malls layer is on by default, so pins and the list appear as soon as the app opens; the Transit layer can be toggled on.

Use Case ID:	#2-2		
Use Case Name:	ClusterMapMarkers		
Created By:	Joseph Tan	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	21/10/2026

Actor	User (included by ViewNearbyVenues (#2-1))
Description	Groups nearby map markers into a single clustered pin at low zoom levels to reduce clutter.
Preconditions	Map is displaying two or more venues close together at the current zoom.
Postconditions	Overlapping markers appear as one clustered pin with a count.
Priority	Medium
Frequency of Use	High
Flow of Events	1. System evaluates on-screen marker positions. 2. Markers that would visually overlap are grouped into one clustered pin. 3. If the user zooms in far enough, the cluster expands into individual pins. 4. If the user taps a cluster, the map zooms into that area.
Alternative Flows	None
Exceptions	None
Includes	None
Special Requirements	None
Assumptions	None
Notes/Issues	Distinct from DeclutterOverlappingPins (#1-4) (repositions venues sharing near-identical real coordinates, at the data layer).

Use Case ID:	#2-3		
Use Case Name:	ToggleMapLayer		
Created By:	Joseph Tan	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	16/09/2026

Actor	User
Description	Shows or hides Transit or Venue pins on the map.
Preconditions	None
Postconditions	The selected layer's pins appear or disappear on the map and matching list.
Priority	Medium
Frequency of Use	High
Flow of Events	1. User taps a layer toggle button. 2. System shows/hides that category's pins and list rows.
Alternative Flows	None
Exceptions	None
Includes	None
Special Requirements	None
Assumptions	None
Notes/Issues	Both layers are off by default.

Use Case ID:	#2-4		
Use Case Name:	RecentreMap		
Created By:	Joseph Tan	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	16/09/2026

Actor	User
Description	Recentres the map on the user's current location.
Preconditions	User has moved the map away from their location.
Postconditions	Map recentres on the user's current GPS location.
Priority	Low
Frequency of Use	Medium
Flow of Events	1. User taps the recentre button. 2. System retrieves current GPS location. 3. Map animates back to centre on it.
Alternative Flows	None
Exceptions	If location is unavailable, an alert is shown.
Includes	None
Special Requirements	Device location services (GPS).
Assumptions	None
Notes/Issues	None

Use Case ID:	#2-5		
Use Case Name:	ViewCrowdSnapshot		
Created By:	Joseph Tan	**Last Updated By:**	Joseph Tan
Date Created:	20/10/2026	**Date Last Updated:**	20/10/2026

Actor	User
Description	Shows an at-a-glance summary of crowds across Singapore at the top of the Areas list.
Preconditions	The venue list has loaded.
Postconditions	The card shows mall counts by crowd level, the crowd level near the user, and the quietest open mall nearby.
Priority	Medium
Frequency of Use	High (every time the app is opened)
Flow of Events	1. User opens the app on the Areas tab. 2. System counts the malls that are quiet, moderate, busy and closed. 3. System finds the crowd level most open malls and stations within 2 km share. 4. System finds the quietest open mall within 5 km. 5. System displays the card above the venue list. 6. User taps the quietest mall, invoking ViewVenueDetail (#3-1).
Alternative Flows	3a. Location unavailable: the "near you" line is hidden. 4a. No open mall within 5 km: the quietest open mall anywhere is shown instead. 4b. Every mall is closed: the card says so instead.
Exceptions	None
Includes	None
Special Requirements	None
Assumptions	None
Notes/Issues	The "near you" line shows a crowd level, not an average percentage, because each mall's percentage is relative to its own peak and can't be meaningfully averaged.

For Functional Requirement #3 – View venue detail

Use Case ID:	#3-1		
Use Case Name:	ViewVenueDetail		
Created By:	Joseph Tan	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	20/10/2026

Actor	User
Description	Displays full crowd detail for a selected venue.
Preconditions	User has selected a venue from the map, list, or search results.
Postconditions	Venue's full detail (crowd %, weekly histogram, opening hours, source) is displayed.
Priority	High
Frequency of Use	High
Flow of Events	1. User selects a venue. 2. System retrieves the venue's full detail. 3. System displays name, category, address, crowd %, crowd level, weekly histogram, and opening hours. 4. If a lower-crowd alternative exists nearby, system invokes ViewAlternatives (#3-2). 5. The crowd level shows Closed whenever the venue's opening hours say it is closed.
Alternative Flows	None
Exceptions	If the venue's data is no longer available, a "not found" message is shown.
Includes	None (extended by ViewAlternatives (#3-2), GetDirections (#3-3))
Special Requirements	None
Assumptions	None
Notes/Issues	On opening, the system refreshes Popular Times for this venue only if its pattern is over a week old. A Directions button invokes GetDirections (#3-3).

Use Case ID:	#3-2		
Use Case Name:	ViewAlternatives		
Created By:	Xu Hanwen	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	21/10/2026

Actor	User
Description	Recommends lower-crowd venues of the same category near the currently viewed venue. Extends ViewVenueDetail (#3-1).
Preconditions	User is viewing a venue's detail screen.
Postconditions	Lower-crowd alternatives, if any, are displayed.
Priority	Medium
Frequency of Use	Medium
Flow of Events	1. System identifies venues of the same category. 2. System filters to those within 3km. 3. System filters to those with a lower crowd percentage. 4. System sorts remaining venues by distance. 5. System displays the results.
Alternative Flows	If the user selects an alternative, ViewVenueDetail (#3-1) is invoked for it.
Exceptions	If no qualifying alternative exists, nothing is shown in this section.
Includes	None
Special Requirements	None
Assumptions	None
Notes/Issues	Transit venues are excluded from alternatives matching.

Use Case ID:	#3-3		
Use Case Name:	GetDirections		
Created By:	Teo Kok Wei, Oscar	**Last Updated By:**	Teo Kok Wei, Oscar
Date Created:	20/10/2026	**Date Last Updated:**	20/10/2026

Actor	User
Description	Shows a route from the user's location to the venue on Crowdy's own map, by public transport, walking, driving or cycling, with the live crowd level at each MRT/LRT station boarded.
Preconditions	User is viewing a venue's detail, and location permission is granted.
Postconditions	The route is drawn on the map with step-by-step instructions.
Priority	Medium
Frequency of Use	Medium
Flow of Events	1. User taps Directions on the venue's detail. 2. System gets the user's current location. 3. System requests a route for the selected mode (public transport by default) from OneMap. 4. System draws the route on the map, coloured by MRT line, and lists the steps. 5. For each MRT/LRT leg, system shows the boarding station's current crowd level. 6. User switches to another mode; steps 3–5 repeat. 7. If the user moves more than 150 m, system recalculates the route.
Alternative Flows	2a. Location permission denied: a message explains that location is needed.
Exceptions	If OneMap is unavailable, system shows "Couldn't load this route right now" with a Try again button.
Includes	None
Special Requirements	OneMap account credentials on the server; device location services.
Assumptions	None
Notes/Issues	Routes are computed by OneMap. A route can occasionally loop round a block near its start or end, because OneMap starts from the nearest road or path it knows.

For Functional Requirement #4 – Crowd heatmap

Use Case ID:	#4-1		
Use Case Name:	ViewCrowdHeatmap		
Created By:	Joseph Tan	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	20/10/2026

Actor	User
Description	Displays an overlay where busy malls and MRT/LRT stations (Moderate and above) glow yellow to red, so hotspots stand out.
Preconditions	User is viewing the map.
Postconditions	The heatmap overlay is shown or hidden on the map.
Priority	Medium
Frequency of Use	Medium
Flow of Events	1. User taps the heatmap toggle. 2. System invokes GenerateHeatmapImage (#4-2) if no fresh image is cached. 3. System places the image on the map, positioned to its real geographic bounds. 4. User taps the toggle again to hide it.
Alternative Flows	None
Exceptions	If image generation fails, the toggle has no visible effect.
Includes	GenerateHeatmapImage (#4-2)
Special Requirements	iOS only, react-native-maps has no gradient/heatmap layer outside Google Maps, which Expo Go no longer supports.
Assumptions	None
Notes/Issues	Quiet, closed and no-data venues contribute no colour.

Use Case ID:	#4-2		
Use Case Name:	GenerateHeatmapImage		
Created By:	Joseph Tan	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	20/10/2026

Actor	System
Description	Renders the crowd heatmap as a single blurred, colour-graded image.
Preconditions	ViewCrowdHeatmap (#4-1) was invoked and no fresh cached image exists.
Postconditions	A PNG image representing current crowd levels across all open venues is available.
Priority	Medium
Frequency of Use	On-demand, cached ~5 minutes
Flow of Events	1. System filters to malls and stations at Moderate crowd level or above. 2. System projects each venue's coordinates onto a canvas. 3. System draws a soft radial colour gradient per venue on a transparent background. 4. System exports the canvas as a PNG.
Alternative Flows	None
Exceptions	None
Includes	None
Special Requirements	Rendered in a headless browser on the server (separate from the signed-in browser used by ScrapePopularTimes (#1-3))
Assumptions	None
Notes/Issues	Stations have no percentage, so each is coloured at the middle of its level's colour band to match its badge.

For Functional Requirement #5 – Search

Use Case ID:	#5-1		
Use Case Name:	SearchVenue		
Created By:	K Eswaran	**Last Updated By:**	Mao Yikai
Date Created:	16/09/2026	**Date Last Updated:**	20/10/2026

Actor	User
Description	Lets the user search the full tracked venue list by name, address or postal code.
Preconditions	None
Postconditions	Matching venues are displayed, best match first.
Priority	High
Frequency of Use	High
Flow of Events	1. User types a venue name. 2. System matches names, address words ('rd' = road) and postal codes, at the start of words only, forgiving one-letter typos in words (not in postal codes). 3. Results are ranked: exact postal code, postal prefix, name prefix, name words, address words, then typo matches. 4. User selects a result, invoking ViewVenueDetail (#3-1).
Alternative Flows	None
Exceptions	If no venues match, a "no results" message is shown.
Includes	None
Special Requirements	None
Assumptions	None
Notes/Issues	Purely client-side against the already-loaded venue list, no separate search API call.

For Functional Requirement #6 – Accounts & Settings

Use Case ID:	#6-1		
Use Case Name:	AccountAuthentication		
Created By:	Teo Kok Wei, Oscar	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	21/10/2026

Actor	User
Description	Let the user sign up for a new account or log into an existing one.
Preconditions	None
Postconditions	User is authenticated and holds a session token.
Priority	Medium
Frequency of Use	Low
Flow of Events	1. User opens Settings and selects Sign Up or Log In. 2. User enters a username and password. 3. System validates credentials. 4. If the user had locally-saved favourites, ManageFavourites (#6-4) is invoked to offer merging them. 5. System returns the user to Settings, now signed in.
Alternative Flows	None
Exceptions	Wrong username/password shows the same generic error either way, so login attempts can't be used to check which usernames exist.
Includes	LoginAccount (#6-2), CreateAccount (#6-3)
Special Requirements	Passwords hashed with bcrypt. Sessions are JWT bearer tokens (30-day expiry).
Assumptions	None
Notes/Issues	Username-based (not email). A deliberate simplification from Lab 1's original plan.

Use Case ID:	#6-2		
Use Case Name:	LoginAccount		
Created By:	Teo Kok Wei, Oscar	**Last Updated By:**	Teo Kok Wei, Oscar
Date Created:	16/09/2026	**Date Last Updated:**	16/09/2026

Actor	User
Description	Allows a user to log in to an existing account.
Preconditions	User has an existing account.
Postconditions	User is authenticated and returned to Settings.
Priority	High
Frequency of Use	Low
Flow of Events	1. User enters username and password. 2. System validates credentials against the database. 3. If correct, system issues a session token.
Alternative Flows	None
Exceptions	If credentials are incorrect, a generic error is shown.
Includes	None
Special Requirements	None
Assumptions	User has already registered.
Notes/Issues	None

Use Case ID:	#6-3		
Use Case Name:	CreateAccount		
Created By:	Teo Kok Wei, Oscar	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	16/09/2026

Actor	User
Description	Allows a visitor to create a new account.
Preconditions	None
Postconditions	A new account is created and stored in the database.
Priority	High
Frequency of Use	Low
Flow of Events	1. User enters a username and password. 2. System validates the username format (3–24 letters/numbers/underscore) and password length (8+ characters). 3. System checks the username isn't already taken. 4. System hashes the password and stores the new account. 5. System issues a session token.
Alternative Flows	None
Exceptions	If the username is already taken, or the input is invalid, an error is shown.
Includes	None
Special Requirements	Password stored only as a bcrypt hash, never in plain text.
Assumptions	None
Notes/Issues	None

Use Case ID:	#6-4		
Use Case Name:	ManageFavourites		
Created By:	Teo Kok Wei, Oscar	**Last Updated By:**	Mao Yikai
Date Created:	16/09/2026	**Date Last Updated:**	20/10/2026

Actor	User
Description	Save or remove venues as favourites, and view them in a dedicated list.
Preconditions	None
Postconditions	The venue is added to or removed from favourites.
Priority	Medium
Frequency of Use	Medium
Flow of Events	1. User taps the heart on a venue row or on the venue's detail screen. 2. If signed in, system saves/removes the favourite against the account. 3. If not signed in, system saves/removes it in local on-device storage. 4. If the user later signs in with local favourites present, they are explicitly prompted (not silently merged) to bring them into the account. The on-device saves are only cleared after the account update succeeds; if it fails, they are kept and the user is asked again at the next sign-in.
Alternative Flows	None
Exceptions	If a favourited venue no longer exists in the data source, it is silently dropped from the list.
Includes	None (extended by AccountAuthentication (#6-1))
Special Requirements	None
Assumptions	None
Notes/Issues	The explicit merge prompt exists to prevent bleeding of one guest's leftover local saves into every subsequent account signed into on the same device.

Use Case ID:	#6-5		
Use Case Name:	ViewSavedVenues		
Created By:	Teo Kok Wei, Oscar	**Last Updated By:**	Joseph Tan
Date Created:	16/09/2026	**Date Last Updated:**	16/09/2026

Actor	User
Description	Displays all venues the user has favourited.
Preconditions	None
Postconditions	A list of saved venues, each with its current crowd level, is displayed.
Priority	Medium
Frequency of Use	Medium
Flow of Events	1. User selects the Saved tab. 2. System retrieves the user's favourited venue ids (account or local, depending on sign-in state). 3. System displays them sorted by distance.
Alternative Flows	None
Exceptions	If no venues are saved, an empty-state message is shown.
Includes	None
Special Requirements	None
Assumptions	None
Notes/Issues	None

Use Case ID:	#6-6		
Use Case Name:	LogOut		
Created By:	Teo Kok Wei, Oscar	**Last Updated By:**	Teo Kok Wei, Oscar
Date Created:	16/09/2026	**Date Last Updated:**	16/09/2026

Actor	User
Description	Ends the current session.
Preconditions	User is signed in.
Postconditions	Session token is discarded; user reverts to guest, on-device-only state.
Priority	Low
Frequency of Use	Low
Flow of Events	1. User selects Log Out in Settings. 2. System discards the stored session token. 3. Saved venues now read from local storage instead of the account.
Alternative Flows	None
Exceptions	None
Includes	None
Special Requirements	None
Assumptions	None
Notes/Issues	Also triggered automatically if the backend ever rejects the session token as expired/invalid.

For Functional Requirement #7 – Language

Use Case ID:	#7-1		
Use Case Name:	ChangeLanguage		
Created By:	Mao Yikai	**Last Updated By:**	Mao Yikai
Date Created:	20/10/2026	**Date Last Updated:**	20/10/2026

Actor	User
Description	Switches the app's text between English and Simplified Chinese.
Preconditions	None
Postconditions	All app text is shown in the chosen language, and the choice is remembered.
Priority	Low
Frequency of Use	Low
Flow of Events	1. User opens Settings. 2. User selects English or 中文. 3. System updates every screen immediately. 4. System saves the choice on the device.
Alternative Flows	None
Exceptions	None
Includes	None
Special Requirements	None
Assumptions	None
Notes/Issues	Venue, street and station names stay in English, since the source data only has English names. OneMap's turn-by-turn steps are translated.

Summary of Use Case Description Changes since Lab 2

Change	Reason
New: SyncPopularTimesCache (#1-5)	Popular Times data is now shared by the team through the database
New: ViewCrowdSnapshot (#2-5)	The home screen opened empty; it now shows a live summary
New: GetDirections (#3-3)	Directions were drawn inside Crowdy instead of handing off to another maps app
New: ChangeLanguage (#7-1)	Support Chinese-speaking users
Updated: #1-1, #1-2, #1-3	Weekly signed-in scraping; fallbacks when a source is unavailable
Updated: #2-1, #3-1, #4-1, #4-2, #5-1, #6-4	Behaviour refined during implementation
"Account & Favourites" renamed "Account & Settings"	It now also holds Change Language

Design Class Diagrams

As the diagrams are very large and span too wide that blurs its contents, please refer to the raw PNG and SVG files that are uploaded together with this document.

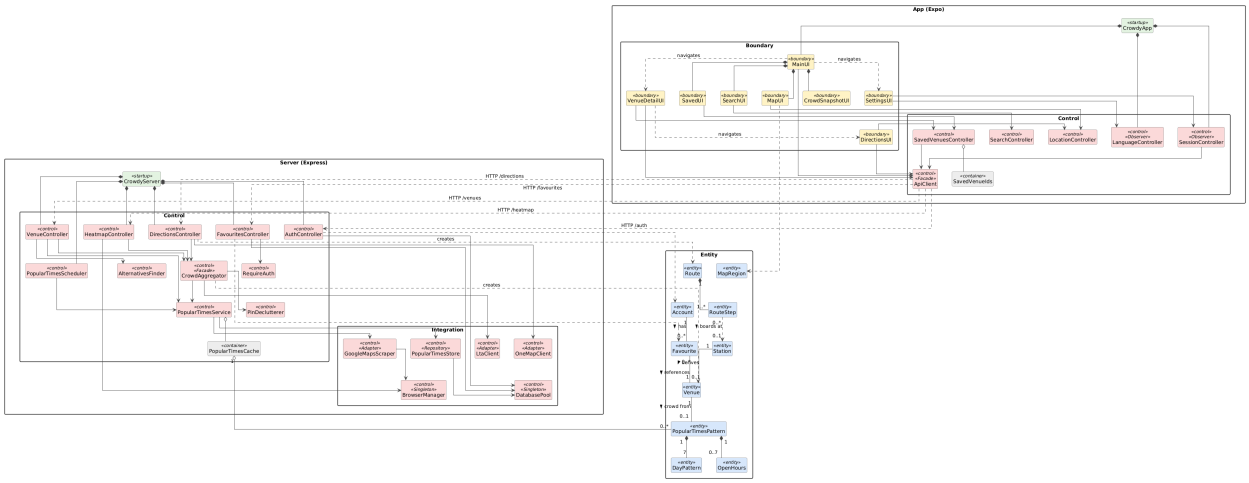


Figure 5. Design Class Diagram: Overview

The design model refines the Lab 2 conceptual model into the classes that make up the implementation. Crowdy is split into an Expo app and an Express server. The app's boundary classes (screens) use app-side control classes, which reach the server only through ApiClient over HTTP. The server's control classes coordinate crowd data, routing, authentication and favourites, and reach external services (LTA DataMall, Google Maps, OneMap and the MySQL database) only through integration classes. Figures 6–8 show each part in detail.

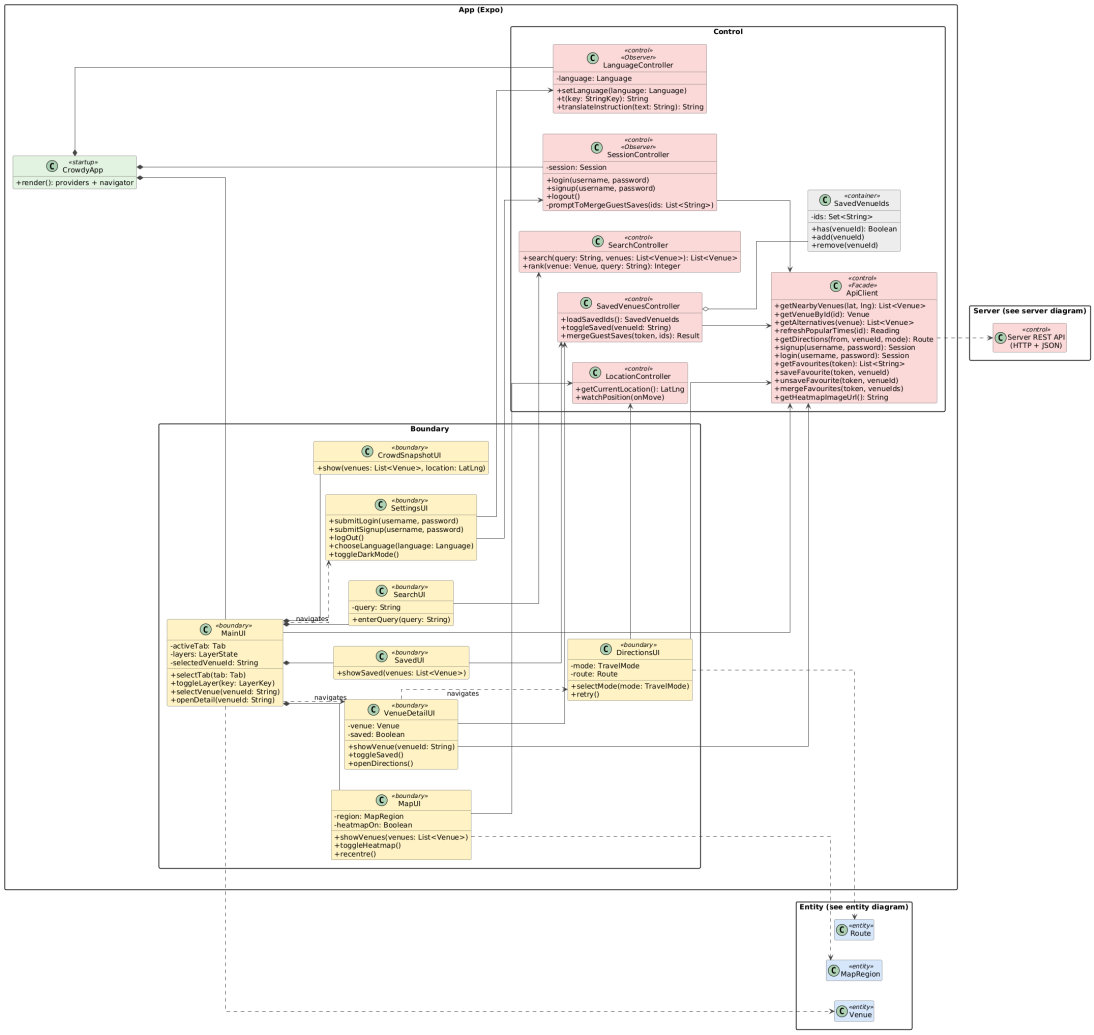


Figure 6. Design Classes: App

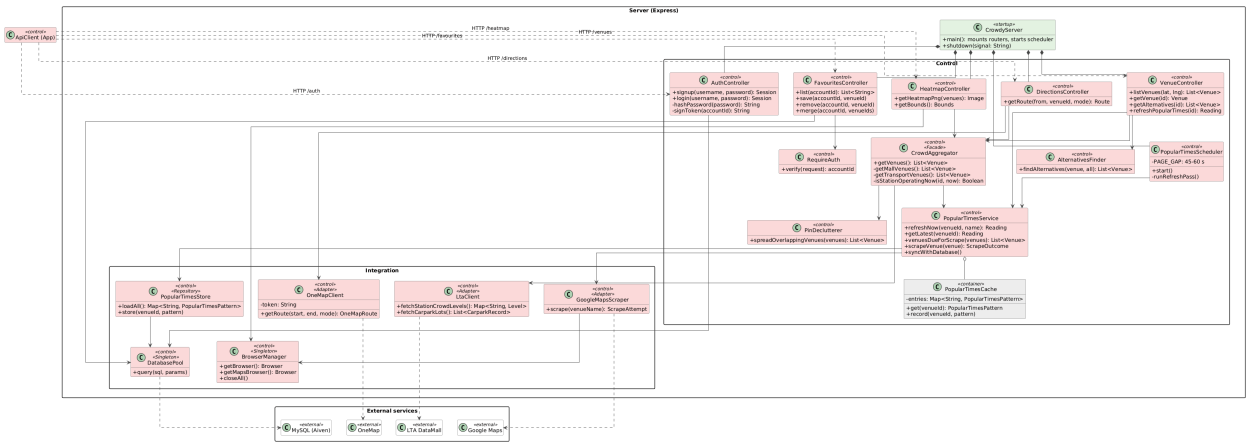


Figure 7. Design Classes: Server

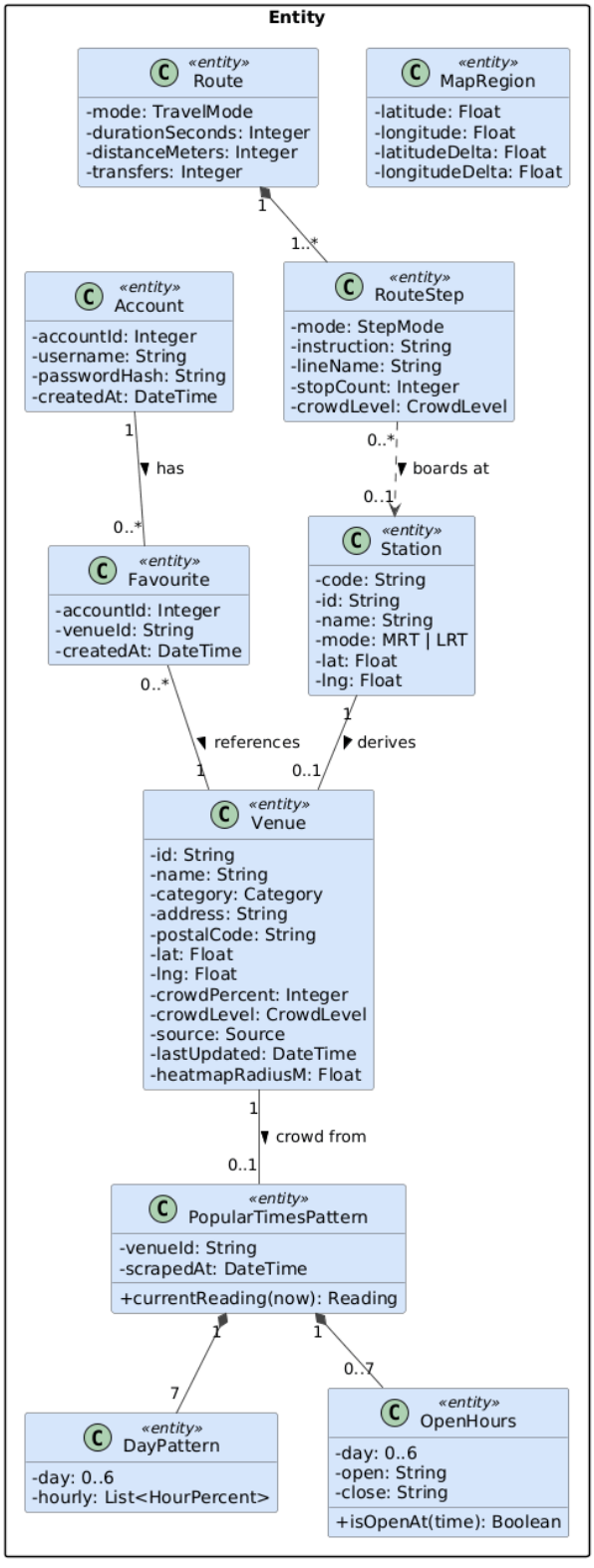


Figure 8. Design Classes: Entities

Changes from the Lab 2 conceptual model (entity class diagram and key boundary/control classes)

Change	Reason
Added start-up classes CrowdyApp and CrowdyServer	Manual heuristic: system initialisation
Lab 2's AuthController and FavouritesController each split into an app-side and a server-side controller (SessionController / AuthController, SavedVenuesController / FavouritesController)	Login and saving happen on both the phone (session, guest saves) and the server (accounts, database)
VenueController moved to the server; CrowdAggregator, PopularTimesService and PopularTimesScheduler added	Crowd data is gathered and cached on the server
New boundaries DirectionsUI and CrowdSnapshotUI, plus new controls DirectionsController, LanguageController and RequireAuth	New use cases: GetDirections (#3-3), ViewCrowdSnapshot (#2-5), ChangeLanguage (#7-1), plus access control
New entities PopularTimesPattern, DayPattern, OpenHours, Route and RouteStep	Popular Times data and directions need their own structures
Integration layer (adapters, store, database pool, browser)	Keeps external services out of the control classes

Dialog Map

You can refer to the raw PNG and SVG files that are uploaded together with this document for better clarity.

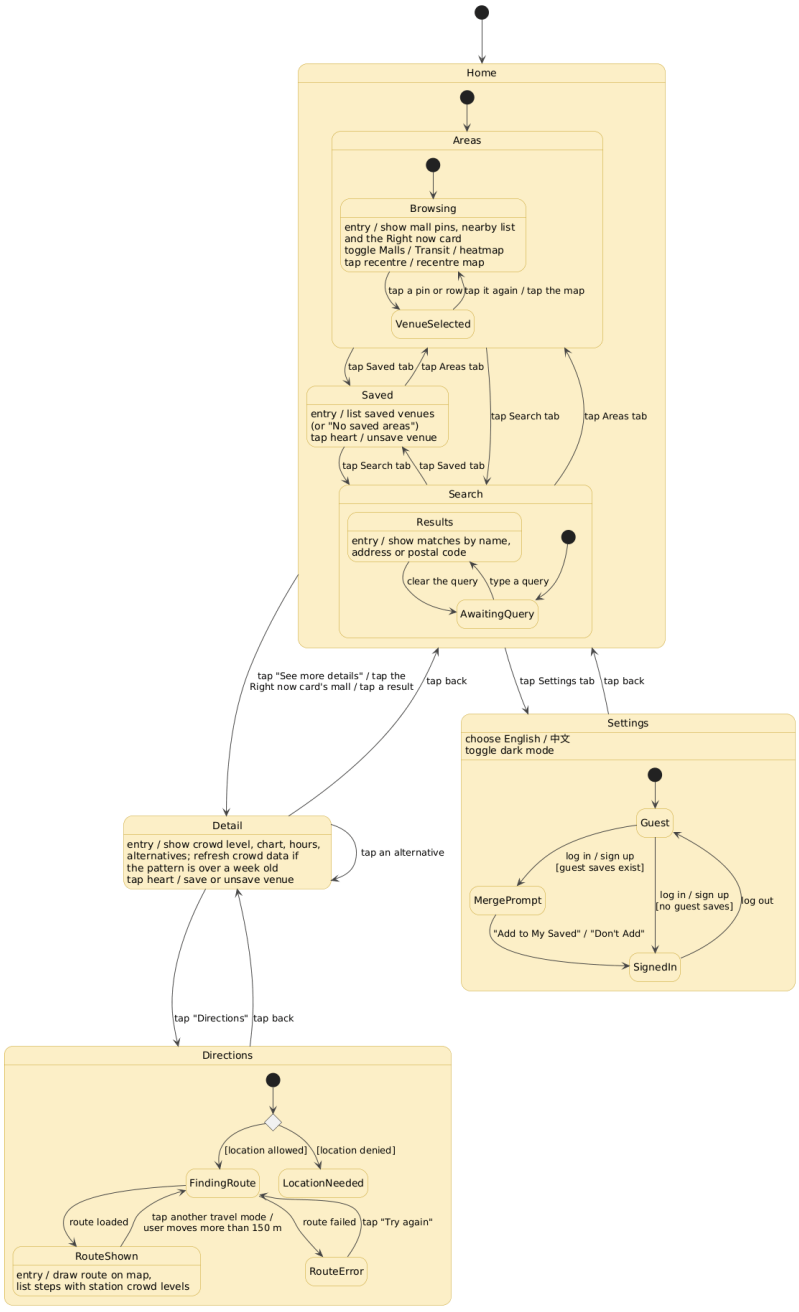


Figure 9. Dialog Map (refined from Lab 2)

The dialog map shows Crowdy's screens as states and the user actions that move between them. Nested states are the modes within a screen (e.g. the Areas, Saved and Search tabs of Home). Text inside a state lists what happens on entering it ("entry /") and actions that stay on the same screen. Transitions are labelled "event [condition]".

System Architecture

Architecture style

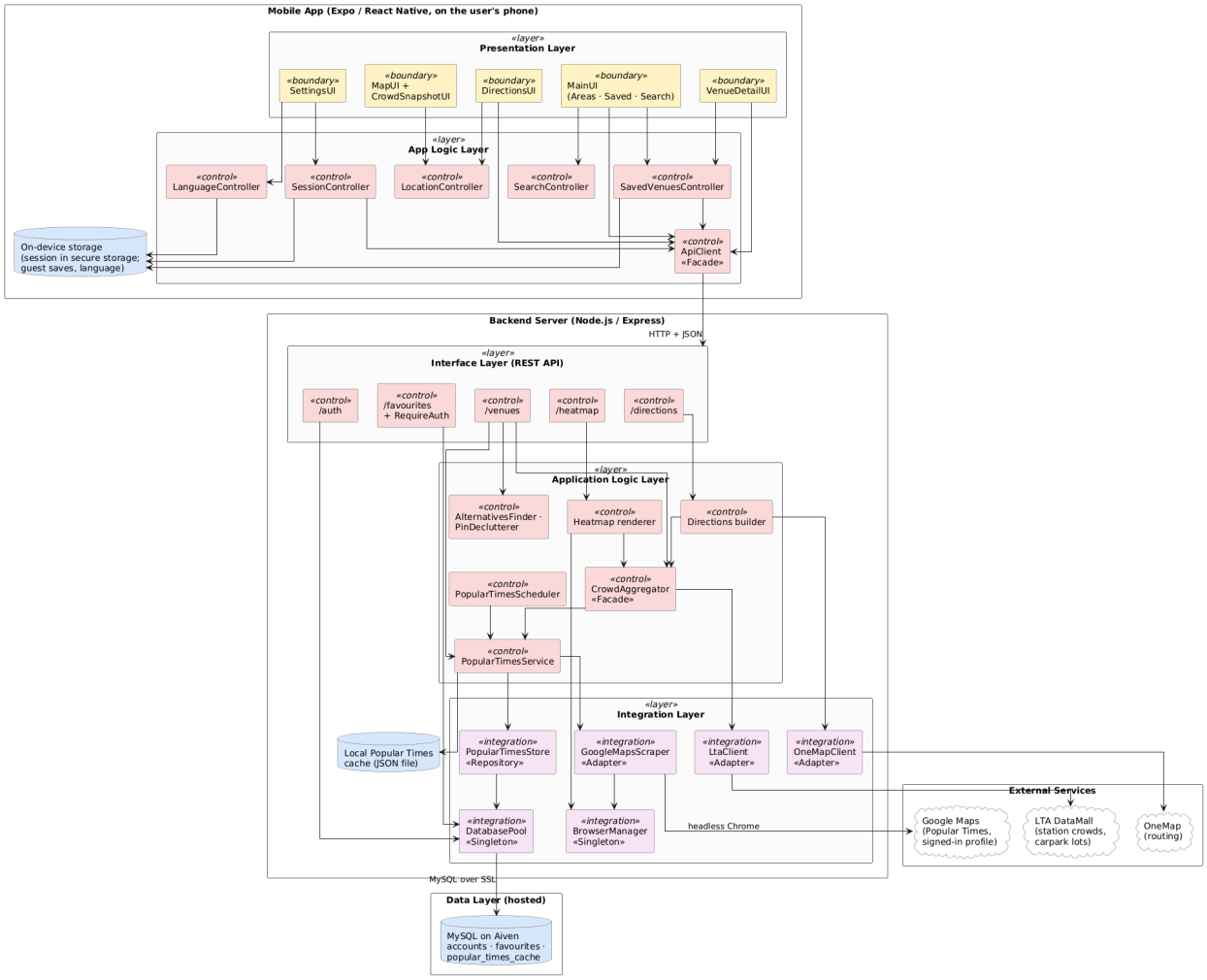


Figure 10. System Architecture

Crowdy uses a client–server, layered architecture. The mobile app is a thin client: it shows data and handles the user's own state (session, saved venues, language), but never contacts an external data source directly. Everything else goes through the backend server's REST API. The server is layered so each layer only calls the one below it: the interface layer (REST endpoints) calls the application logic layer, which calls the integration layer, which is the only layer that talks to the database and external services. This keeps external services swappable (e.g. replacing the Popular Times source only changes one adapter), keeps API keys and credentials on the server, and lets the whole team share one set of crowd data.

Layers and Components

Presentation layer (app). The screens the user interacts with:

- MainUI: the home screen, with Areas, Saved and Search tabs.
- MapUI and CrowdSnapshotUI: the crowd map, heatmap and Right now card.
- VenueDetailUI: a venue's crowd level, chart, hours and alternatives.
- DirectionsUI: routes to a venue.
- SettingsUI: account, language and dark mode.

App logic layer (app). Controls each screen's behaviour:

- SessionController: login, sign-up and logout, plus the guest-saves prompt.
- SavedVenuesController: saving and unsaving venues, on the device or in the account.
- SearchController: ranks venues by name, address and postal code.
- LocationController: the phone's GPS.
- LanguageController: English/Chinese text.
- ApiClient: the single point of contact with the server, a Facade.

Small personal data (session token in secure storage; guest saves and language) is kept in on-device storage.

Interface layer (server). REST endpoints, one per resource:

- /venues, /directions and /heatmap are public.
- /auth handles login and sign-up.
- /favourites is protected by RequireAuth, which checks the user's sign-in token.

Application logic layer (server):

- CrowdAggregator: merges station and mall crowd data into one venue list, a Facade.
- PopularTimesService and PopularTimesScheduler: keep mall patterns fresh weekly.
- AlternativesFinder and PinDeclutterer: quieter nearby venues, and separating overlapping pins.
- Heatmap renderer: draws the crowd heatmap image.
- Directions builder: turns OneMap routes into steps with station crowd levels.

Integration layer (server). The only code that talks to the outside world:

- LtaClient, GoogleMapsScraper and OneMapClient (Adapters) convert each external service's format into Crowdy's own.
- BrowserManager (Singleton) shares one headless browser.
- PopularTimesStore (Repository) and DatabasePool (Singleton) handle the database.

The server also keeps a local Popular Times cache file, so it keeps working if the database is unreachable.

Data layer: A MySQL database hosted on Aiven stores accounts, favourites and the team's shared Popular Times cache, over an SSL connection.

External services:

- LTA DataMall: live MRT/LRT platform crowding and carpark lots.
- Google Maps: Popular Times, read through a headless browser signed in to a project account.
- OneMap: routing for public transport, walking, driving and cycling.

LTA and OneMap are called over HTTPS.

Technology choices and justification

Technology	Used for	Why
Expo + React Native (TypeScript)	Mobile app	One codebase for iOS and Android; Expo Go lets the team test on real phones without app-store builds
Node.js + Express (TypeScript)	Backend server	Same language as the app, so types such as Venue are shared and teammates can work on both sides; Express is simple for a REST API
MySQL on Aiven (free tier)	Accounts, favourites, shared crowd cache	Relational data with clear links (account → favourites); hosted for free, so every teammate's backend shares one database
Puppeteer (headless Chrome)	Popular Times; heatmap rendering	Google has no official Popular Times API; the same browser also renders the heatmap image
LTA DataMall	Station crowds, carpark lots	Official Singapore government data, free, real-time
OneMap	Directions	Free Singapore government routing with public transport; no billing account needed (Google's routing requires one)
JWT + bcrypt	Sign-in	Industry-standard token login, with passwords stored only as hashes
react-native-maps	Map display	The standard map component for React Native; uses Apple Maps on iOS

Application Skeleton

The application skeleton is the full source code in our team repository. We submit our in-progress prototype, which already implements the use cases above. The code follows the design model: each class in the design class diagram maps to a file. Key classes and public methods are documented with JSDoc comments, the TypeScript equivalent of Javadoc.

Repository: http://10.96.189.23/sc2006/scsj-h3

Folder structure

```
crowdy/
├── app/                          Mobile app (Expo / React Native)
│   ├── App.tsx                   Start-up class (CrowdyApp)
│   ├── src/
│   │   ├── screens/              Boundary: Home, Detail, Directions, Settings
│   │   ├── components/           Boundary: map, venue rows, Right now card,
│   │   │                         directions sheet, crowd chart, badges
│   │   ├── context/              Control: session (login state)
│   │   ├── i18n/                 Control: language switch, English/Chinese text
│   │   ├── services/             Control: ApiClient (api.ts), search, saved venues,
│   │   │                         secure session storage
│   │   ├── theme/                Light/dark colours
│   │   ├── navigation/           Screen routes
│   │   └── types/                Entity: Venue, Route, MapRegion, ...
│   └── tests/                    Automated tests (search, saves, translation)
│
└── server/                       Backend server (Node.js / Express)
    ├── src/
    │   ├── index.ts              Start-up class (CrowdyServer)
    │   ├── routes/               Control: REST endpoints (/venues, /directions,
    │   │                         /heatmap, /auth, /favourites, /lines)
    │   ├── middleware/           Control: RequireAuth (access control)
    │   ├── services/             Control + integration: Popular Times, scheduler,
    │   │                         LTA, OneMap, heatmap, database, auth
    │   ├── data/                 CrowdAggregator (venues.ts) and fixed data:
    │   │                         malls, stations, train hours, rail lines
    │   └── types/                Entity: Venue, Route, ...
    ├── scripts/                  One-off tools: database setup, mall list,
    │                             station hours, Google sign-in
    └── tests/                    Automated tests (auth, favourites,
                                  alternatives, opening hours)
```

The project is split into the mobile app and the backend server, matching the client–server architecture described in System Architecture.

In the app, screens and components are the boundary classes, the context, i18n and services folders hold the control classes, and types holds the entity classes.

In the server, routes and middleware form the REST interface layer, services holds the application logic and integration classes, and data holds the crowd aggregation logic and fixed venue data. Each side has its own automated tests (npm test).

Credentials such as API keys and the database password are kept in a .env file on the server and are not committed to the repository.